

①

Machine learning Techniques in Data Science Lecture - VII

Introduction

At the same time when John Von Neumann published his famous work of automata theory and followed by realization of computing machine, scientist started thinking of whether it is possible to make a computing machine intelligent like a human being a new area emerges which we know today as Artificial Intelligence or AI in short. Machine learning is a part of artificial intelligence. To describe a brief history we mention the following important discovery —

* 1950s:

— Samuel's checker-playing program.
(Arthur Samuel of IBM)
He coined the term machine learning

* 1960s:

- Neural network Rosenblatt's Perceptron
- Pattern Recognition
- Minsky & Papert prove limitations of Perceptron.

* 1970s:

- Symbolic concept induction
- Expert systems and knowledge acquisition bottleneck
- Quinlan's ID3
- Natural language processing (symbolic)

* 1980s:

- Advanced decision tree and rule learning
- Learning and planning and problem solving.
- Resurgence of neural network

- Valiant's PAC learning theory
- Focus on experimental methodology.

* 90's ML and Statistics

- support Vector Machines.
- Data Mining.
- Adaptive agents and web applications.
- Text mining
- Reinforce learning
- Ensembles
- Bayes Net learning.

* 2000':

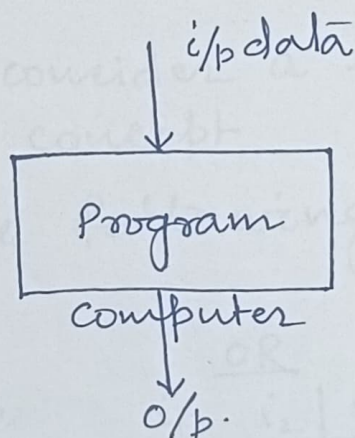
- Deep learning or deep structured learning.

- Applications:
- ① self-driving cars.
 - ② Deep blue won against the world chess champion Garry Kasparov.
 - ③ Human Vision surpassed by ML system.
 - ④ Machine translation systems using Neural networks.

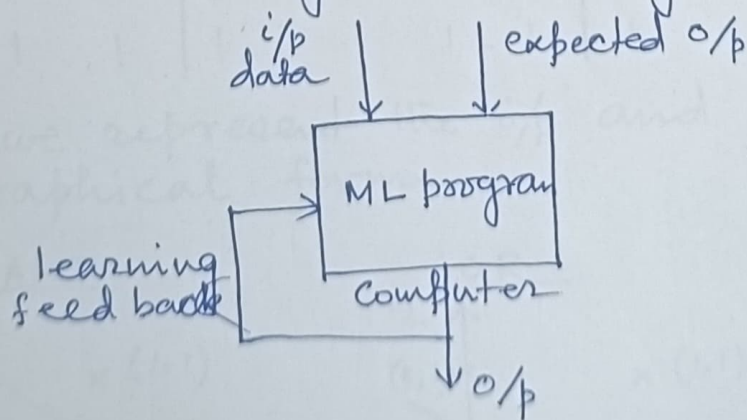
The astonishing growth of machine learning becomes feasible due to the advancement of Hardware, memory technology and the improvement of software technology as well.

today even in a laptop we have GPUs having 512 no of processing unit and very large memory unit within affordable cost. Similarly availability of object oriented programming semantic net based data base management system in the s/w area makes the artificial intelligence at large and machine learning specifically available to the common man. After the brief historical introduction let us try to understand the framework of machine learning.

In conventional programming the framework can be represented schematically as —



while in machine learning, there are two phases, namely — training and testing.



During the training phase we give example data as well as the corresponding expected output as the i/p. The ML program after processing them generates the o/p. Now it calculates an error —

$$E = |\text{Expected o/p} - \text{generated o/p}|$$

This error is fed back to the program and adjust various parameters of the program ~~to until~~ until the error get minimized.

Once the training is complete the program becomes ready to take unknown i/p and generates the correct o/p. The last phase is called a test phase.

Let us now consider a simple example to clarify the concept

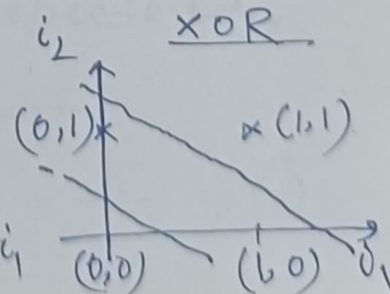
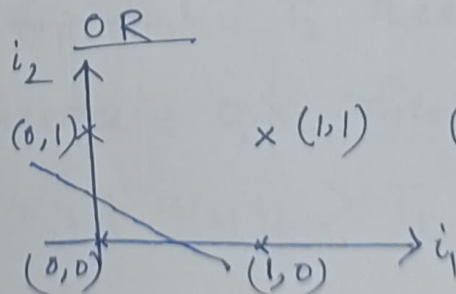
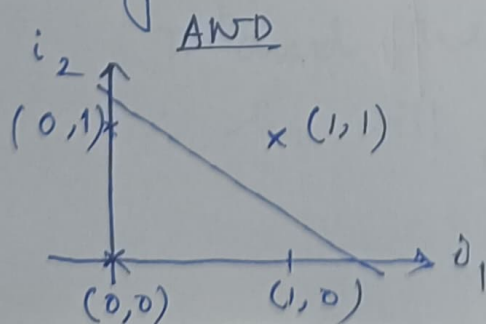
consider the following truth tables:

AND		
i_1	i_2	o/p
0	0	0
0	1	0
1	0	0
1	1	1

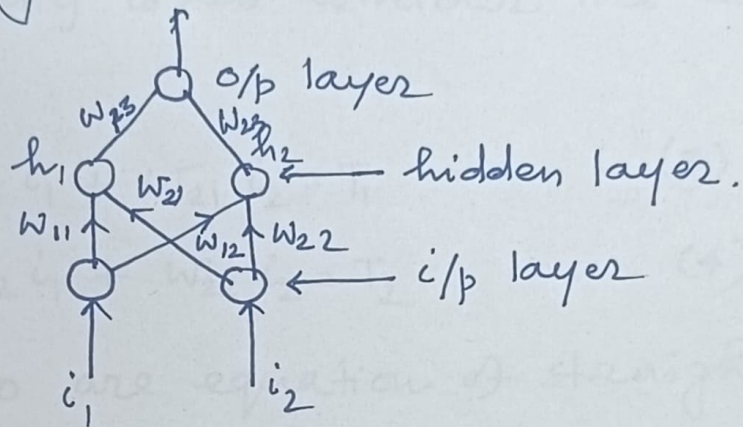
OR		
i_1	i_2	o/p
0	0	0
0	1	1
1	0	1
1	1	1

XOR		
i_1	i_2	o/p
0	0	0
0	1	1
1	0	1
1	1	0

now we represent the i/p and o/p relations in graphical form —



In case of XOR to separate the true o/p from the false o/p we need to draw two separator lines which can easily be obtained using a multi layer perceptron.



step 1: Initialize the weights w_{ij} with random no.

step 2: The i/p to the hidden node h_1 is —

$$w_{11}i_1 + w_{21}i_2$$

and the i/p to the hidden node h_2 is

$$w_{12}i_1 + w_{22}i_2$$

The nodes h_1 and h_2 will generate o/p for which will enter as i to the next layer as the i/p when the i/p to h_1 and h_2 are greater than or equal to a threshold value. Let the threshold value for h_1 is T_1 and that for h_2 is T_2 respectively.

$\therefore h_1$ will generate o/p when —

$$w_{11}i_1 + w_{21}i_2 \geq T_1 \dots (1)$$

and h_2 will generate an o/p when

$$w_{12} i_1 + w_{22} i_2 \geq T_2 \dots \dots (2)$$

For brevity let's consider the equality case.

$$\text{i.e. } w_{11} i_1 + w_{21} i_2 = T_1 \dots \dots (3)$$

$$\text{and } w_{12} i_1 + w_{22} i_2 = T_2 \dots \dots (4)$$

These two are equation of straight lines like —

$$i_1 = - \frac{w_{21}}{w_{11}} i_2 - \frac{T_1}{w_{11}} \dots \dots (5)$$

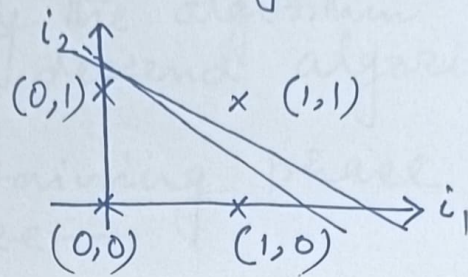
$$\text{and } i_2 = - \frac{w_{22}}{w_{12}} i_1 - \frac{T_2}{w_{12}} \dots \dots (6)$$

The gradient of the first straight line is $-\frac{w_{21}}{w_{11}}$ and the intercept is $-\frac{T_1}{w_{11}}$

Similarly the gradient of the second straight line is $-\frac{w_{22}}{w_{12}} i_1$ and the intercept is $-\frac{T_2}{w_{12}}$

Since they are generated using random values of the weights therefore they may not give the adequate separation between the true or false cases.

For example after the first iteration the straight lines may take the form as—

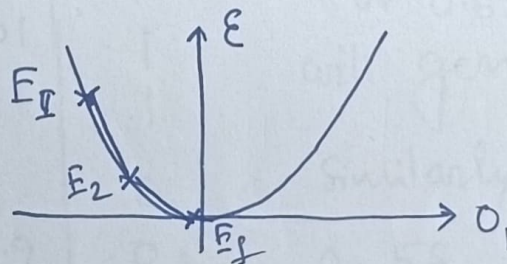


The corresponding o/p generated at the o/p node is 0, say and let $\hat{o}_1$ be the expected o/p—

Then the error generated is—

$$E = (o_1 - \hat{o}_1)^2 \quad \text{---(7)}$$

This is the equation of a parabola and the error curves looks like—



Suppose

Step-4: Suppose with the initial random value of the output the initial value of the error is E_1 . Then the error E_1 is fed back to the lower layer (hidden layer) and the weights w_{13} , w_{23} , w_{12} , w_{12} , w_{21} and w_{22} is such in such a way that in the next iteration becomes E_2 . The same process is repeated till it reaches the final minimum value of the error E_F .

The decrease of the error is followed the path as the gradient of the error parabola. That is why the algorithm is known as the gradient descend algorithm.

During Training phase the i/p data looks like —

i_1	i_2	Expected o/p
0	0	0
0.1	0.1	0
0.15	0.15	0
⋮	⋮	⋮
0	1	1
0.1	0.8	1
0.2	0.5	1
⋮	⋮	⋮
1	0	1
0.9	0.01	1
0.85	0.02	1
⋮	⋮	⋮
1	1	0
0.9	0.9	0
0.8	0.8	0

After the training is over if we give an input set —

0.13 0.25 it will generate

a ~~zero~~ '0' o/p

or 0.63 0.89 then it will generate the o/p '0'

Similarly for

0.58 0.11 it will generate the o/p '1'.